A Unified Driver–Compute Framework for General-Purpose Computation on Edge Devices

A Case Study in Biomedical Sensing

Kommana Sri Krishna Ramesh
23ECB0B19

Bibek Chaudhary
23ECB0F33

Abhinav Magesh
23ECB0F25

Team B19

September 2, 2026

Motivation

- Biomedical edge devices are siloed: pulse oximeters, ECG monitors, BP cuffs, glucometers each use a different transport and data format.
- Serial (USB), Bluetooth Low Energy (BLE GATT), proprietary USB – no common interface.
- Integrating each device today means writing a bespoke driver *and* a bespoke pipeline every time.
- **Goal:** one general software framework that can connect to *any* edge device and compute *anything* on its data.

Problem Statement

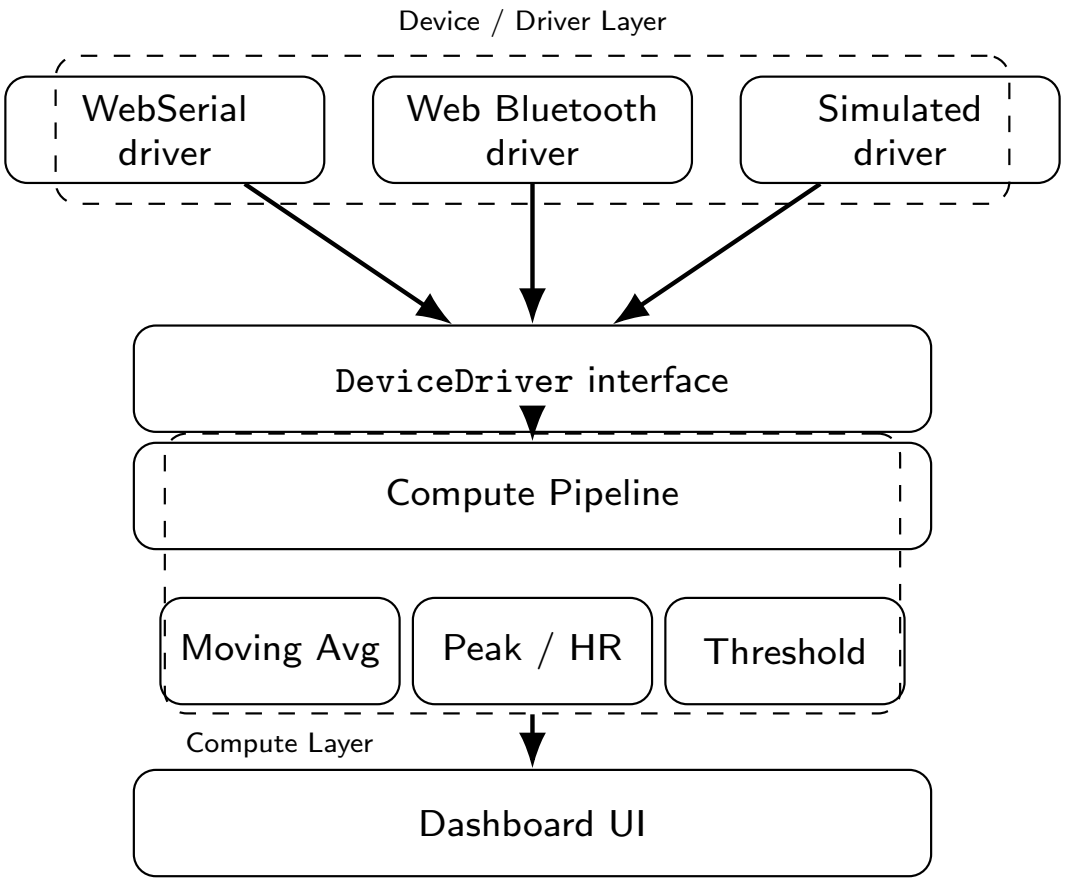
Given devices $D = \{d_1, \ldots, d_n\}$ over transports $T(d_i) \in \{\text{serial}, \text{BLE}, \text{USB}\}$ emitting readings $(t, \text{channel}, \text{value})$, build a system where:

- Adding a **new device** = implementing one driver interface, no other changes.
- Adding a **new computation** = implementing one compute-module interface, no driver changes.
- The system is **always demonstrable**, even with zero hardware attached.

Related Work

- **IEEE 11073 / Bluetooth GATT health profiles** – standardize data *within* a device class, not across heterogeneous classes/transports.
- **EdgeX Foundry, Node-RED** – general IoT gateway platforms, but need separate server infrastructure.
- **WebSerial / Web Bluetooth / WebUSB** – browser APIs for direct hardware access; underused as a *general-purpose* device integration layer. We treat the browser tab itself as a universal, zero-install gateway.

Proposed Architecture



The `DeviceDriver` Interface

Every device integration implements the same contract:

- `isSupported()` – browser/hardware capability check
- `connect()` / `disconnect()`
- `getStatus()` – disconnected / connecting / connected / error
- `onData(callback)` – stream of {`timestamp`, `channel`, `value`, `unit`}

The dashboard only ever talks to this interface – it never knows whether data is coming from real hardware or the simulator.

Three Driver Types Built

- **WebSerial driver** – generic USB-serial device, line-delimited `channel,value` readings.
- **Web Bluetooth driver** – standard BLE **Heart Rate Service (0x180D)**, works with real commercial BLE straps/watches.
- **Simulated driver** – synthetic ECG / SpO_2 / HR / BP generator with selectable scenarios (normal, tachycardia, desaturation). Always available, default on load – the demo never dead-ends.

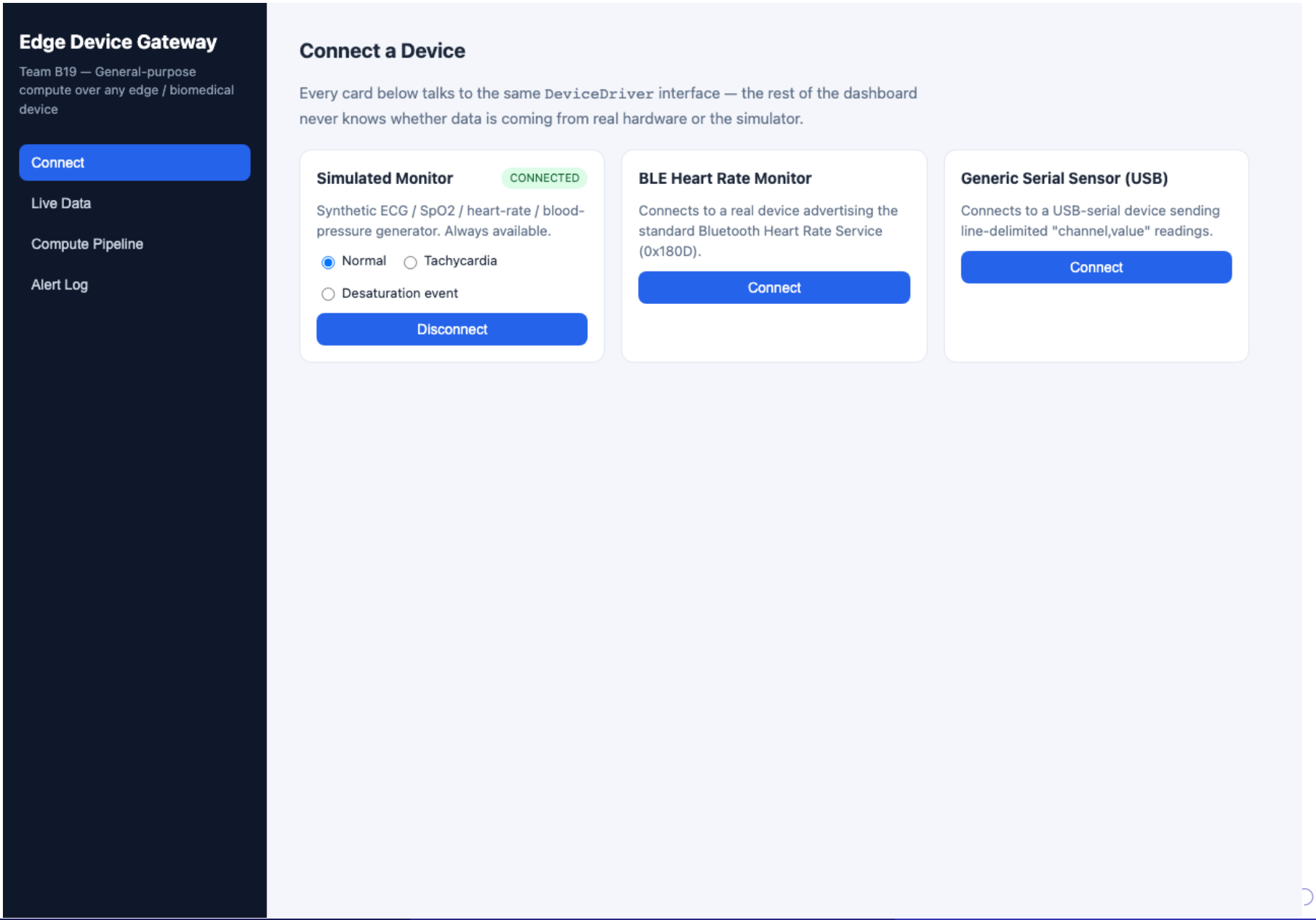
Compute Pipeline Concept

- Every incoming reading, from *any* driver, is fanned through an ordered list of pluggable `ComputeModules`.
- A module can emit **derived readings** (e.g. a filtered value, a computed heart rate) and/or **alert events**.
- Modules can be enabled/disabled and configured live from the dashboard.

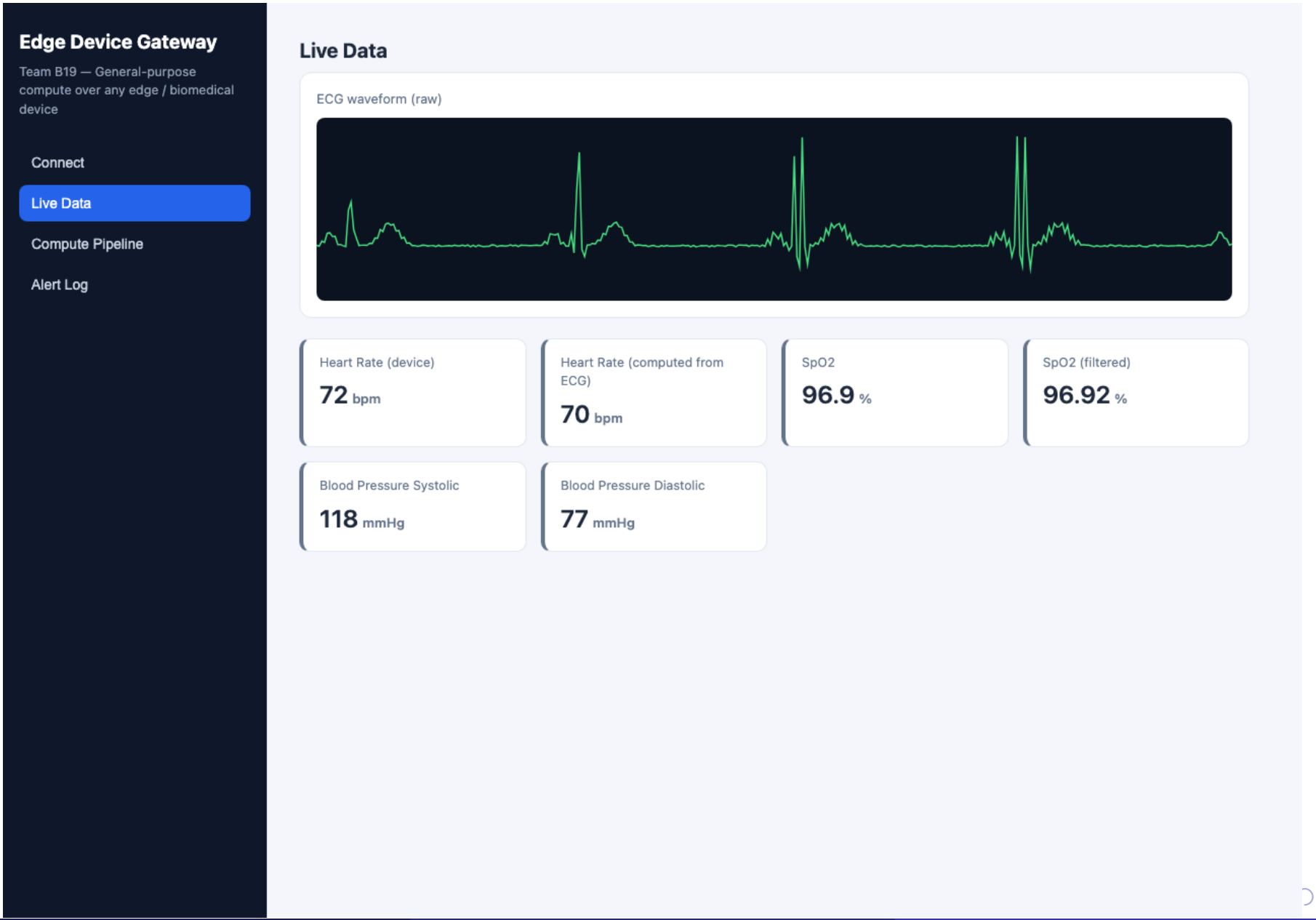
Compute Modules Built

- **Moving-average filter** – smooths a noisy channel (e.g. SpO_2).
- **ECG peak / heart-rate detector** – threshold-crossing R-peak detection with a refractory period; derives instantaneous BPM live from the raw waveform.
- **Threshold alerting** – configurable per-channel warning/critical thresholds; emits alerts on severity transitions (no duplicate spam).

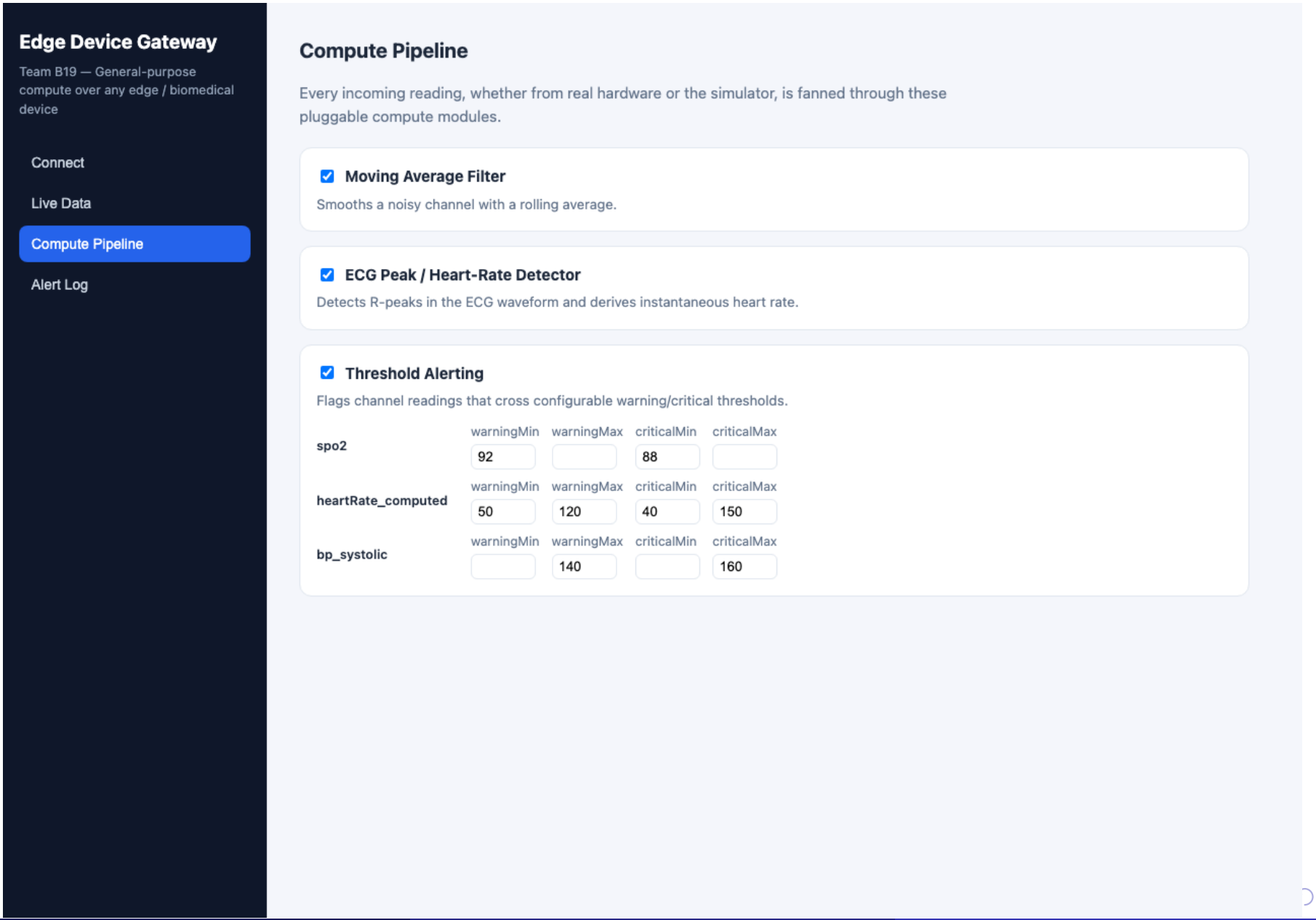
Dashboard: Connect



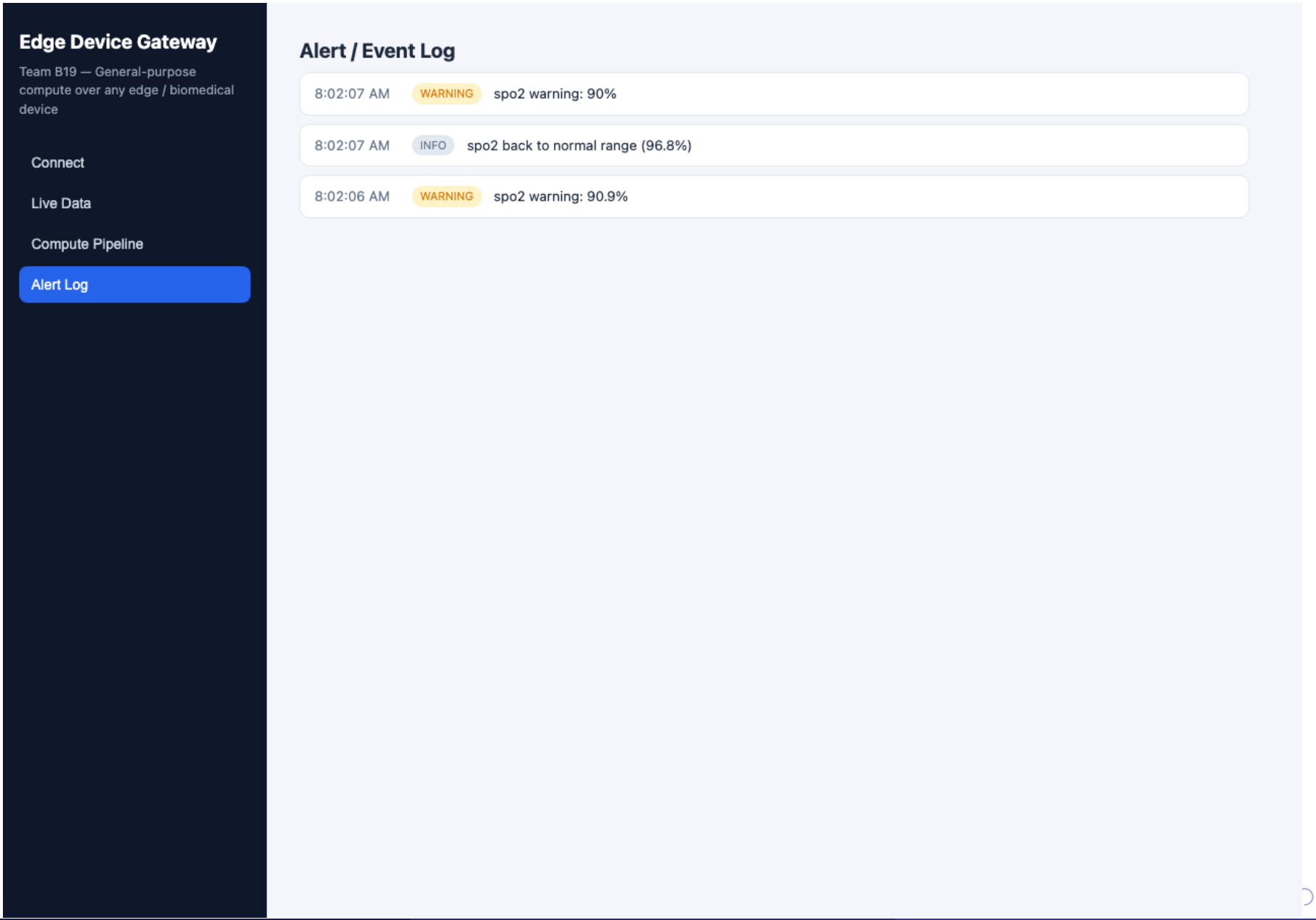
Dashboard: Live Data



Dashboard: Compute Pipeline



Dashboard: Alert Log



Use Case Walkthrough

1. Simulated monitor connects automatically – live ECG, SpO_2, HR, BP.
2. Switch scenario to **Desaturation event**.
3. Moving-average filter smooths the noisy SpO_2 channel.
4. SpO_2 crosses the warning threshold (92%) $\rightarrow$ alert logged.
5. Same pipeline, zero code changes, would fire identically on a real pulse oximeter.

Extensibility & Future Work

- **WebUSB driver** – interface already accommodates it; deferred as lower priority.
- **Clinical-grade signal processing** – e.g. Pan–Tompkins QRS detection.
- **Persistence & multi-device trends** – optional storage layer.
- **Native companion apps** – for browsers without WebSerial/Web Bluetooth.

Conclusion

- A driver–compute architecture generalizes edge device integration: transport-specific acquisition, transport-agnostic computation.
- Validated with a working, client-only prototype connecting to *real* hardware (WebSerial, Web Bluetooth) and a simulator, with zero backend.
- New device = one driver. New analysis = one compute module. Independently.

Team B19

- Kommana Sri Krishna Ramesh – 23ECB0B19
- Bibek Chaudhary – 23ECB0F33
- Abhinav Magesh – 23ECB0F25

Thank you